How the Install Tabs Install

The install tabs of 44helper put software on the node one step at a time. This summary covers how each step is defined, how the page sends it, how the helper checks and runs it, what a person does by hand, and which tab installs what.

ASL3 DVSWITCH SVXLINK M17 AMPERSAND

SUMMARY · READ-ONLY REVIEW

What each install tab installs

The tab list is in code (the `TABS` list). Each tab with an install has its own step table and its own purge table. Some tabs (Overview, Services, Ports, Boot, Config) do not install software.

TAB	INSTALLS	INSTALL STEPS	PURGE STEPS	DISTROS
ASL3	ASL3 (Asterisk and app_rpt), Allmon3 and Cockpit	16	11	Bookworm and Trixie (same lists)
DVSwitch	DVSwitch Server, Analog_Bridge, MMDVM_Bridge, md380-emu, the digital-mode gateways, and Cockpit	13 (Bookworm), 15 (Trixie)	12 (both)	Bookworm and Trixie
SVXLink	SvxLink from the package, or a USRP build of it with its own service; and Cockpit	8 (package, both distros), 12 (USRP)	12 (each)	Bookworm, Trixie, USRP
M17	USRP2M17 through USRP2M17-Selector, with its web page; and Cockpit	10	12	Standard
Ampersand	amp-server (radio node) or amp-hub (conference hub)	7 (server), 6 (hub)	9 (server), 10 (hub)	Trixie only (checked by the first step)
Cloudflare	cloudflared, as a token connector or a quick test tunnel	5 (token), 3 (quick)	8 (token), 6 (quick)	Standard
Claude Remote	Node.js 22 and Claude Code, run as a service for remote control	8	7	One list ("claude")
Update	Package maintenance, the optional Webmin, Ajenti and fail2ban, and package recovery	11 steps in one table (Part 5.9)	The optional tools have remove alternatives	Standard

The counts are the lengths of the step tables in the code. Every purge table ends with the same clean-up family (Part 5.0).

The Router Install, Full Tunnel and Split Tunnel tabs set up tunnels rather than installing packages. They are covered in Stage 2. System Optimization applies and reverts settings rather than running steps (Part 5).

What a step is

Every step is a record in the helper's own table, with an id, a number, a title and a command. The page only shows the records and sends an id back. The command itself is never trusted from the page.

2.1 The fields of a step

FIELD	MEANING
`id`, `num`, `title`	The identifier, the step number, and the heading shown on the page.
`cmd`	The exact command for a fixed step.
`cmd_template` and `fields`	For steps that need a value (a callsign, a port, a node number). The template contains `{{field}}` markers.
`timeout`	Seconds allowed. Package, build and script installs have no timeout.
`note`, `warning`	Text shown before the step is run.
`manual` or `manual_block`	The page offers no Run button for the step, and it is meant for a terminal (Part 7). The server does not check this flag.
`alts`	Alternative steps that belong to the same heading (Update tab).

2.2 Input rules for fields

Each field must match a fixed pattern before the command is built. The rules are in `_STEP_FIELD_RULES`. A field with no rule, or a value that fails its rule, is refused with the rule's wording:

FIELD	ACCEPTED VALUE
callsign	3 to 16 letters, digits or hyphen
dmrid	6 to 9 digits
port, rx_port	1 to 5 digits
branch	Letters, digits and . _ / - (up to 100)
username	1 to 32 letters, digits and . _ -
version	An 8-digit release date
node	4 to 7 digits
hubpassword	Up to 64 characters from a fixed set with no spaces, quotes, slashes or ampersands. Blank means a private node.
authmode	2 to 16 letters, for example OPEN
allowlist	Comma-separated 4 to 7 digit node numbers, or blank
cruser, crfolder, password	A Linux login name (lowercase, up to 32); a folder path of up to 200 characters with no spaces; 1 to 64 characters from the hub password set (other passwords go through the Custom box)

From the Run button to the result

1. **The page loads the tab's steps.** The step tables are served as JSON from `/api/<tab>/script` (for example `/api/asl3/script`). The page shows the steps, their commands and their notes.
2. **The page sends the step.** A POST goes to `/api/<tab>/action` with the mode (install, purge or custom), the distro, the step id, any field values, and the exact command text the page is showing.
3. **The helper rebuilds the command.** It looks the step up by id, fills the template with the field values after checking them, and compares the result with the command text it was sent. If they differ, the step is refused with "This step's command was changed on the page, so it was not run."
4. **Root and state gates run.** The request must come from a logged-in session. For installs and apt commands, two gates run next: the masked-service gate (Part 6) and the package-health gate (Part 6).
5. **The command starts as a job.** Package changes start as a transient systemd unit. Other commands start as a background process in their own process group. Details are in Part 4.
6. **The page follows the job.** It polls `/api/action_status?job_key=...`. The result carries the output so far, a status, and a success flag.

3.1 Why the command text must match

The page sends back the command it showed the person. The helper runs only the command it built from its own table, and refuses if the two differ. A changed command in the browser, or a stale page after an update, cannot run something the table does not hold. The Custom box is the one exception (Part 6.3).

3.2 Job keys

Each job has a key, such as `asl3_install_bookworm_prereqs` (tab, mode, distro and step id). Running the same step again while it is still running returns the running job rather than starting a second one.

Package changes and everything else

4.1 Two ways to run

KIND OF COMMAND	HOW IT RUNS	TIMEOUT
Package change (apt install, remove, upgrade, purge and similar)	A transient systemd unit named `44helper-job-<key>`, started with `systemd-run`. The job keeps running if the helper restarts. On return, the helper re-attaches to the unit and shows its output.	None
Other command	A background shell in its own process group. Output is read line by line. Killed in two steps if it times out: a terminate, then a kill after a short wait.	The step's timeout (900 seconds by default)

Builds (compiling from source) and installer scripts have no timeout either, because they can take many minutes on a Pi Zero 2 W.

4.2 Quiet mode for some steps

Steps marked `quiet_eligible` (for example the Update tab's fix-missing and reconfigure steps) can run with quiet mode on. Quiet mode pauses the node's services first, so the package job does not compete for the SD card. If quiet mode cannot start, the package job does not run.

4.3 Output and results

- Job output is kept up to a cap of 64 KB.
- Package progress lines from apt and dpkg are removed from the output before it is shown.
- A job's status is one of running or done, with a success flag and a return code.

What each tab's steps do

The lists below follow the step tables in the code, in order. An (M) marks a manual step. (Optional) steps are marked in their titles.

5.0 Shared pieces

- **Cockpit.** The ASL3, DVSwitch, SVXLink and M17 install tabs each end with two steps that install the Cockpit web admin UI (900 seconds) and enable its socket. Each purge ends with an optional "Stop and purge Cockpit" step, which is not run in a normal purge pass because another tab may still rely on Cockpit, and its confirmation warns that a Cockpit web terminal will cut off.
- **The purge clean-up family.** Every purge table has four clean-up steps after the removal: purge packages that were removed but not purged, unmask the tab's own services, remove broken boot links for the tab's services, and a read-only report of what is "still left behind" (60 seconds).
- **The Restore point.** The ASL3, DVSwitch and M17 install tabs end with a step "Save original configs as Restore point". It should be run only after all the systems the operator will use are installed and set up. It saves the config files of every installed system to the library once, never overwriting saved files, and running it later adds only systems installed since. The Restore point, save points and presets are managed on the Config tab.

5.1 ASL3 (16 install steps)

1. Install prerequisites
2. Download the ASL3 repository package
3. Install the ASL3 repository package
4. Refresh the package index
5. Install ASL3 core (900 s)
6. Install Allmon3
7. Install Cockpit (900 s)
8. Enable the Cockpit socket
9. Install the node database and text-to-speech utilities
10. (Optional) The full ASL3 appliance branding and firewall defaults (900 s)
11. Remove the default Allmon3 account
12. Set the Allmon3 admin password (fields: username, password)
13. Enable the Allmon3 service
14. Enable the node database updater timer
15. **Configure node settings (asl-menu)** (M, Part 7)
16. Save the original configs as the Restore point

Purge (11 steps): stop and disable the services, purge the ASL3 and Allmon3 packages, remove unused dependencies, remove the repository package, clean up the downloaded repo package, the four clean-up steps, then (optional) remove leftover node config and (optional) purge Cockpit.

5.2 DVSwitch

- **Bookworm (13 steps):** download the DVSwitch repository-install script, run it (it adds the apt source and key), refresh the index, install DVSwitch Server (base, Analog_Bridge, md380-emu and MMDVM_Bridge), enable the services at boot, enable the digital-mode gateways, "Start DVSwitch after the network is ready" (the netcheck), install and enable Cockpit, set the callsign, DMR ID and BrandMeister password (fields), two optional web-server clash steps (turn the DVSwitch dashboard off, or move it to another port, with a `port` field), and the Restore point.
- **Trixie (15 steps):** the same, with a read-only "check what is already here" first and a "test install (changes nothing)" dry run (120 s) before the real install.
- **Purge (12 steps):** remove 44helper's netcheck and boot-order settings, stop and disable the services, purge the packages, remove unused dependencies, remove the apt repository, the four clean-up steps, then optional steps for leftover config, leftovers from the old third-party build script, and Cockpit.

5.3 SVXLink

- **Package (8 steps):** refresh the index, install SvxLink Server (900 s), install the ALSA utilities, add the svxlink user to the audio and serial groups, set the callsign (field), enable and start SvxLink (60 s), then install and enable Cockpit.
- **USRP fork, built from source (12 steps):** a read-only conflict check, build dependencies (900 s), clone the USRP fork (a branch field), create the svxlink user, configure and build (compiling takes several minutes, with no time limit), a read-only check that the USRP part was built, a read-only locate of the installed files, switch UsrpLogic on and set its ports (fields), install and start the svxlink-usrp service, an optional callsign change, then Cockpit.
- **Purge (12 steps, the same list for all three):** stop and disable SvxLink (package and USRP build), purge the package, remove unused dependencies, the four clean-up steps, then optional removal of leftover node config, the USRP binaries and config under `/usr/local`, the svxlink-usrp unit, the cloned source tree, and Cockpit.

5.4 M17 (10 install steps)

1. Install git, sudo and the build tools (900 s)
2. Install PHP for Apache
3. Clone USRP2M17-Selector
4. Run its installer, which builds USRP2M17 (3600 s; callsign field)
5. Add the M17 node to rpt.conf, modules.conf and extensions.conf (node number field)
6. Restart Asterisk
7. Save the original configs as the Restore point
8. Install a sudoers rule for the web UI
9. Install Cockpit
10. Enable the Cockpit socket

Purge (12 steps): undo the Asterisk config edits, restart Asterisk, stop and disable USRP2M17, remove the web UI, remove the sudoers drop-in, remove the unit, the four clean-up steps, then optional removal of the clone directory and Cockpit.

5.5 Ampersand

- **Server (7 steps):** a read-only conflict check (30 s), the required packages, the udev HID rule for COS and PTT access, download and unpack the server (version field, the 8-digit release date), (optional, M) a first run as a foreground smoke test, install and start the service (run-as user and web port fields), and (optional) a web UI password.
- **Hub (6 steps):** a conflict check, the required packages, download and install the hub (version field), set the hub's node number, node password (blank means a private node with no ASL registration), IAX port (4569, or 4568 if ASL3 already uses 4569), auth mode (OPEN means no checks) and allowlist, (optional, M) other settings by hand in `/usr/etc/amp-hub.env`, then enable and start the hub.
- **Purge:** server 9 steps and hub 10 steps (stop, remove the unit, the binary or program files, the service account for the hub, the staged tarball, the four clean-up steps, and optional config removal).
- Both need Debian 13 (Trixie). The conflict check runs first and stops on any other release.

5.6 Cloudflare

- **Token connector (5 steps):** a read-only pre-flight with a WireGuard guardrail, install cloudflared from the Cloudflare apt source (900 s), install the connector service with the token pasted by hand (M), verify the connector (read-only), and map a public hostname and an Access policy in the Cloudflare dashboard (M).
- **Quick test (3 steps):** the same pre-flight and install, then run a temporary tunnel in the foreground by hand (M, Part 7).
- **Purge:** token 8 steps (uninstall the connector service, remove the package, remove the apt source and signing key, the four clean-up steps, and optional removal of stored credentials); quick 6 steps (the package and apt source, then the clean-up steps).

5.7 Claude Remote (8 install steps)

1. Check what is already here (read-only)
2. Install Node.js 22, pinned to a version and checked against its SHA-256 hash (900 s)
3. Install Claude Code (900 s)
4. Sign in as the login user, a manual step in a terminal (M, Part 7)
5. Create the Remote Control folder (login user and folder fields)
6. Start Claude Remote as a service
7. Check the service
8. Stop Claude Remote

Purge (7 steps): stop and remove the service, remove Claude Code, remove Node.js 22, then the four clean-up steps.

5.8 Cockpit

Cockpit is not a tab of its own in these tables. It is installed by the closing steps of the four install tabs above (Part 5.0), and its purge is an optional last step of each purge.

5.9 Update

The Update tab runs maintenance, not an installation. Its 11 steps are kept in one table, and some have alternatives under them.

STEP	WHAT IT DOES
1 Refresh package index	`apt update`
2 Fix missing or broken packages	Finishes interrupted installs (`dpkg --configure -a`), then `apt install -f`. Quiet mode can be used. It changes nothing when nothing is broken. Alternatives: a check whether apt is still running (read-only), and a narrower reconfigure of half-installed packages.
3 Upgrade packages	With one alternative, the full upgrade, which may install or remove packages.
4 Autoremove unused packages	With one alternative that also purges their config.
5 Clean the local package cache	With one alternative, autoclean, which keeps recent packages.
6 to 8 (Optional) Webmin, Ajenti 2, fail2ban	Each has an install step and an alternative (remove). fail2ban has two alternatives, including a read-only status step.
9 to 11 Package recovery 1, 2, 3	Check for broken packages (read-only), finish half-installed packages, and re-install broken packages. Other tabs send the operator here when `dpkg --audit` reports problems.

What stops a step from running

6.1 Masked services

A masked service cannot start, and a package install that needs it fails. Before an install step, the helper tries to clear the masks on that tab's own services (`_guard_autofix`). If any stay masked, the step is refused and the message names them, with the command to unmask them by hand: `sudo systemctl unmask`.

6.2 Broken package system

Before a package step, the helper runs `dpkg --audit`. If it reports problems, the step is refused, and the message sends the operator to the Update tab's package recovery steps (9, 10 and 11). If `dpkg --audit` does not answer within 60 seconds, the step is allowed. The check applies to package-changing commands only, and six Update-tab steps are exempt so they can repair the problem: fix missing packages, `dpkg --configure`, the apt-running check, and the three recovery steps.

6.3 The Custom box

The Custom box runs any command text, as root, after the same login check. Package commands in it still pass the masked-service gate. It is not restricted to the step table. The book should describe it as a root shell for commands, so it is used with care.

6.4 Root

Every install action first checks that the helper is running as root. A helper not run as root refuses the step.

6.5 Login

The install actions need a logged-in session. Five failed logins from one address within five minutes lock that address out for 15 minutes.

Steps done by hand

Four steps are marked `manual` and three are marked `manual_block`. The page shows them with their notes and offers no Run button. The server does not check the flag, so a request that sends one of these commands would still start it. Each one is interactive or needs a person, so do them by hand.

STEP	WHY IT IS MANUAL
Configure node settings (asl-menu), ASL3	An interactive full-screen menu. The helper has no way to send it keystrokes.
Ampersand first run, amp-server	Runs in the foreground until Ctrl-C.
Ampersand hub settings file	Optional settings in `/usr/etc/amp-hub.env`, edited by hand.
Cloudflare token service	The token is pasted by hand, so it does not pass through a command.
Cloudflare dashboard mapping	The hostname and access policy are set in the Cloudflare web dashboard.
Cloudflare quick test	Runs in the foreground.
Claude sign-in	Claude shows a sign-in link to open on another device. It runs in a terminal (Stage 5 covers the terminal).

What the install tabs show

8.1 Commands are checked, not trusted

Every step's command comes from the helper's table, and the page's copy is only compared with it. The book can describe the install tabs as safe against a changed page, while noting the Custom box as the exception.

8.2 Long jobs survive a restart

A package install runs as a systemd unit, so a restart of the helper does not stop it. A shell command does not have that protection.

8.3 Distro checks are per tab

Some tabs accept only one release. Ampersand needs Trixie, M17 has one standard path, and SVXLink has a USRP option. The book should list the release for each tab before the install steps.

8.4 Steps are a fixed list

The steps are written into the file. A new step, or a changed command, needs a new version of 44helper. That keeps the install behaviour fixed for each release, and it means the book can describe each step.

8.5 Not checked

The vendor scripts that some steps download (the DVSwitch repository script, the USRP2M17-Selector installer, the Ampersand packages and the Cloudflare packages) were not read. This summary describes the steps the helper runs, not what those scripts do inside.